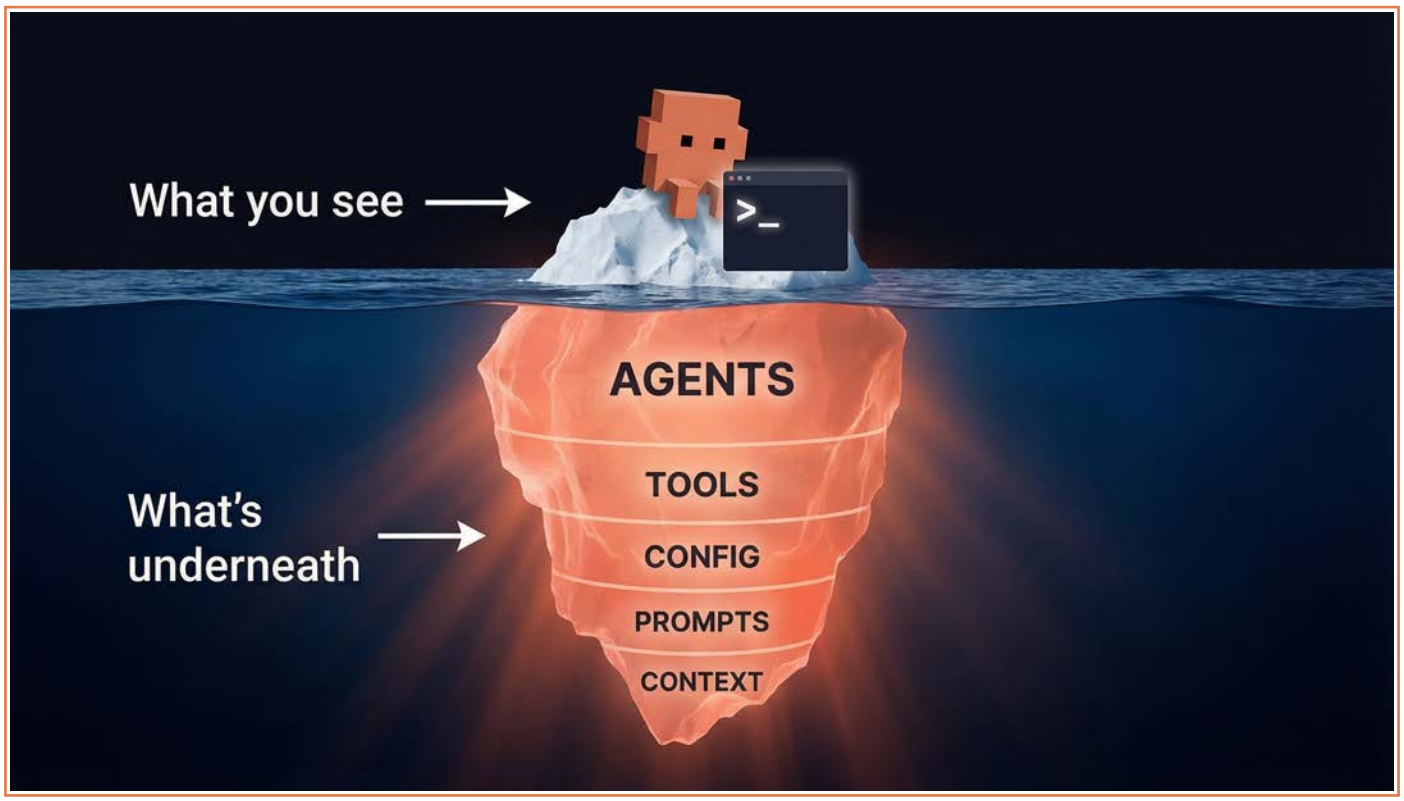


CLAUDE CERTIFIED ARCHITECT

Study Guide

Every concept from the official Anthropic certification exam guide, broken down with visual diagrams and plain English summaries.

Domain	Weight
Agentic Architecture & Orchestration	27%
Tool Design & MCP Integration	18%
Claude Code Configuration & Workflows	20%
Prompt Engineering & Structured Output	20%
Context Management & Reliability	15%

Created by Mark Kashef · Mar 2026

HOW TO USE THIS GUIDE

Study one domain at a time

Each domain has its own section divider. Work through one section per study session.

Read the diagram first

Every concept has a visual diagram. Study the diagram before reading the bullets -- your brain will retain the structure.

Focus on the takeaway boxes

The coral-topped boxes at the bottom of each page contain the single most important idea. If you remember nothing else, remember that.

Use the exam tips

Where included, exam tips tell you exactly how Anthropic tests each concept. They flag the traps.

Print the 5 Rules cheat sheet

The final page is a standalone cheat sheet. Print it and keep it visible while you study.

Test yourself with the interactive prompts

Use the community-created study prompts (linked in Resources) to have Claude quiz you domain by domain.

CONTENTS

3 The 5 Domains Overview

4 DOMAIN 1: AGENTIC ARCHITECTURE
- 5 The Agentic Loop
- 6 Hub-and-Spoke Architecture
- 7 Prompts vs Hooks

8 DOMAIN 2: TOOL DESIGN & MCP
- 9 Tool Descriptions
- 10 MCP Scoping
- 11 Tool Overload

12 DOMAIN 3: CLAUDE CODE CONFIG
- 13 Configuration Hierarchy
- 14 When to Use What
- 15 CI/CD Integration

16 DOMAIN 4: PROMPT ENGINEERING
- 17 Few-Shot Advantage
- 18 Guaranteed JSON
- 19 Validation Loop

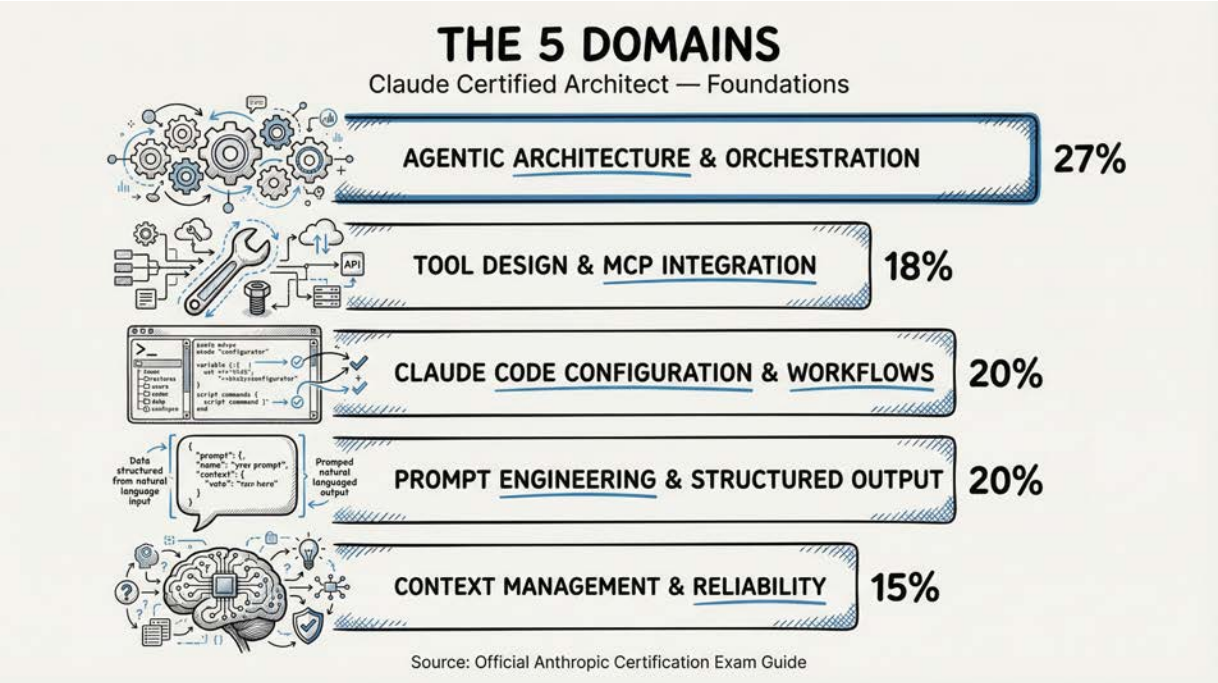
20 DOMAIN 5: CONTEXT MANAGEMENT
- 21 Context Window Problem
- 22 When to Escalate
- 23 Error Propagation

24 The 5 Rules Cheat Sheet
25 Resources

OVERVIEW

The 5 Domains

Anthropic's certification is built around these 5 core areas.



What You Need to Know

- Agentic Architecture (27%) covers how agents loop, coordinate with subagents, and enforce rules with hooks vs prompts.
- Tool Design & MCP (18%) covers how Claude connects to external systems and how tool descriptions determine routing.
- Claude Code Configuration (20%) covers CLAUDE.md hierarchy, skills, commands, plan mode, and CI/CD.
- Prompt Engineering (20%) covers few-shot examples, structured output with JSON schemas, and validation loops.
- Context Management (15%) covers the 'lost in the middle' effect, escalation patterns, and error propagation.

TL;DR

The cert covers a lot of ground. This guide distills each domain into the concepts that matter for both the exam and real-world usage -- so you get the 80/20 without reading 40 pages.

★ If you only study one domain, study Agentic Architecture. It's 27% of the exam and the foundation for everything else.

DOMAIN 1

AGENTIC ARCHITECTURE

How agents think, coordinate, and enforce rules

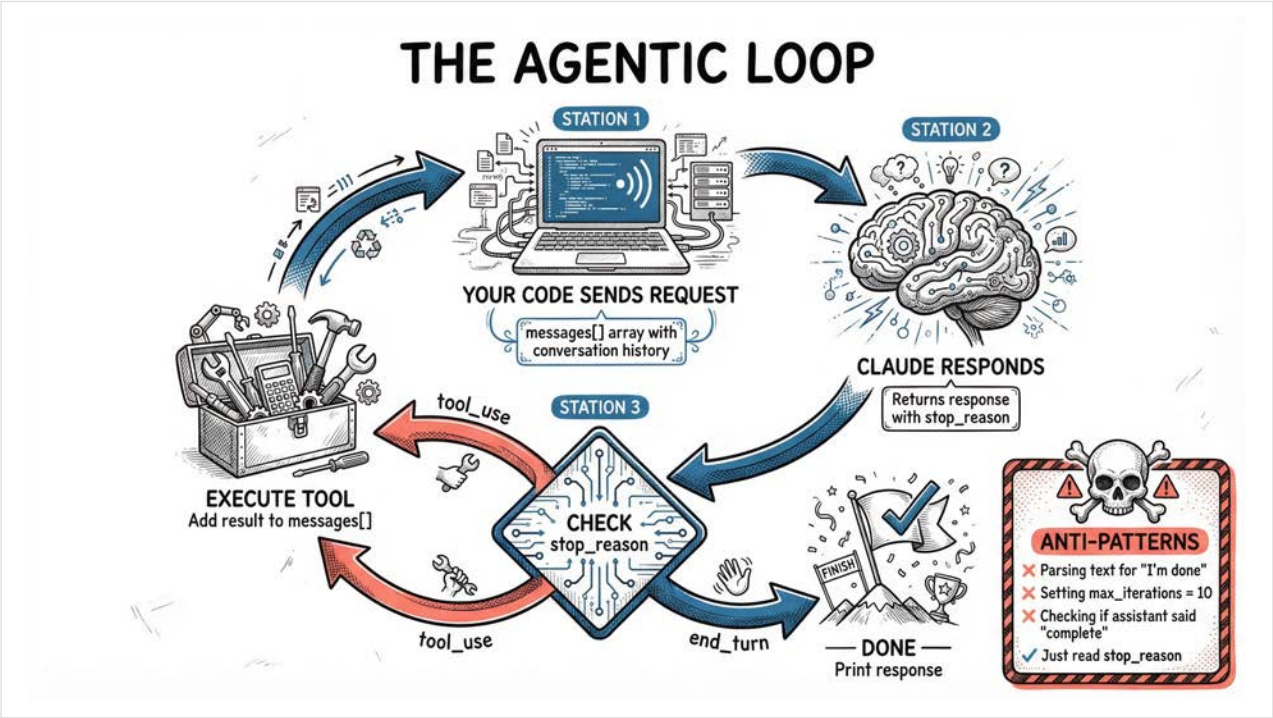
27%

of the exam

DOMAIN 1: AGENTIC ARCHITECTURE (27%)

The Agentic Loop

The engine that powers every Claude agent.



What You Need to Know

- Your code sends a request. Claude responds. You check the stop_reason field.
- If stop_reason is 'tool_use': execute the tool, add the result back, send again.
- If stop_reason is 'end_turn': Claude is done. Print the response.
- Anti-pattern: don't parse text for 'I'm done.' Don't set arbitrary iteration limits. Don't check if Claude said 'complete.'
- Just read stop_reason. That's the only reliable signal.

> **TL;DR**
>
> *The agentic loop is the engine under every Claude workflow you've ever used. Understanding it makes every other concept in this guide click into place.*

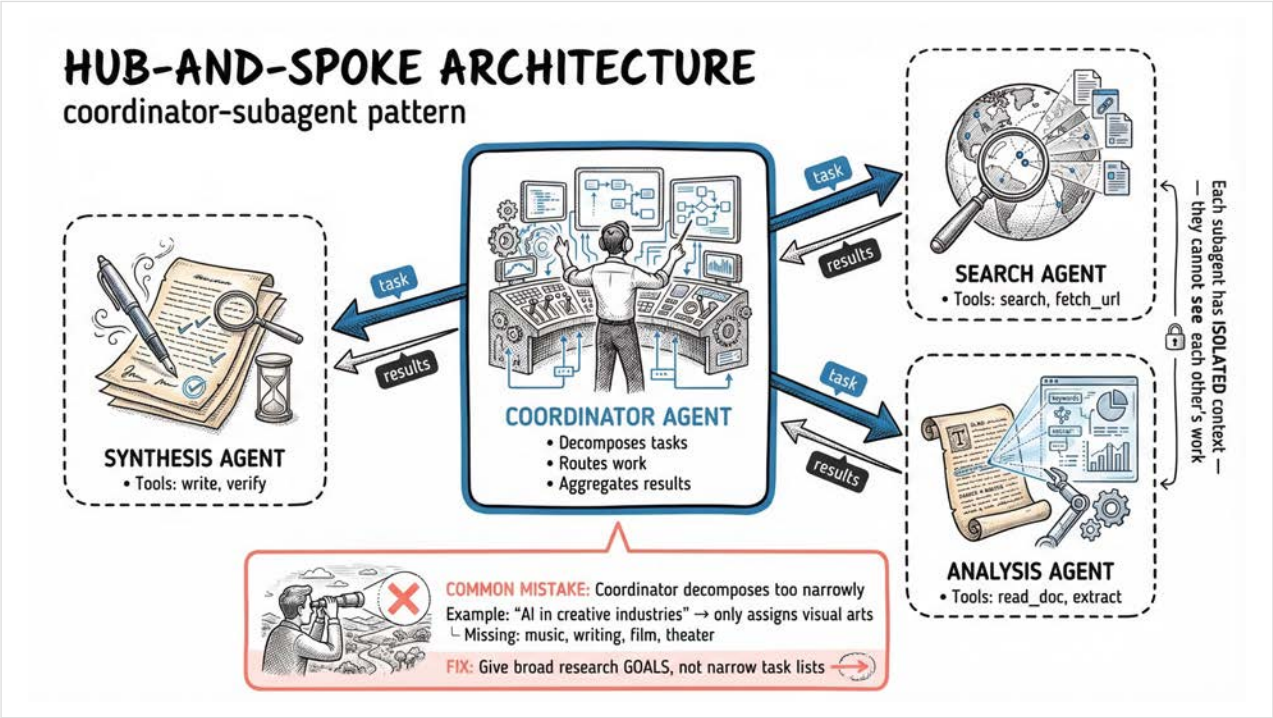
★ The loop is: send request, check stop_reason, execute tool or stop. Nothing else.

Exam Tip: The exam tests whether you know the three anti-patterns. Memorize them.

DOMAIN 1: AGENTIC ARCHITECTURE (27%)

Hub-and-Spoke Architecture

The coordinator-subagent pattern for complex tasks.



What You Need to Know

- One coordinator breaks tasks down, delegates to specialized subagents, combines results.
- Each subagent has isolated context. They cannot see each other's work.
- The coordinator must explicitly pass information between subagents.
- Common mistake: coordinator decomposes too narrowly, missing entire categories.
- Fix: give broad research goals, not narrow checklists.

> **TL;DR**
>
> *Think of it like a bad manager with a great team. Each specialist executes perfectly in isolation. The coordinator's job is to scope broadly enough and synthesize at the end -- not micromanage the steps.*

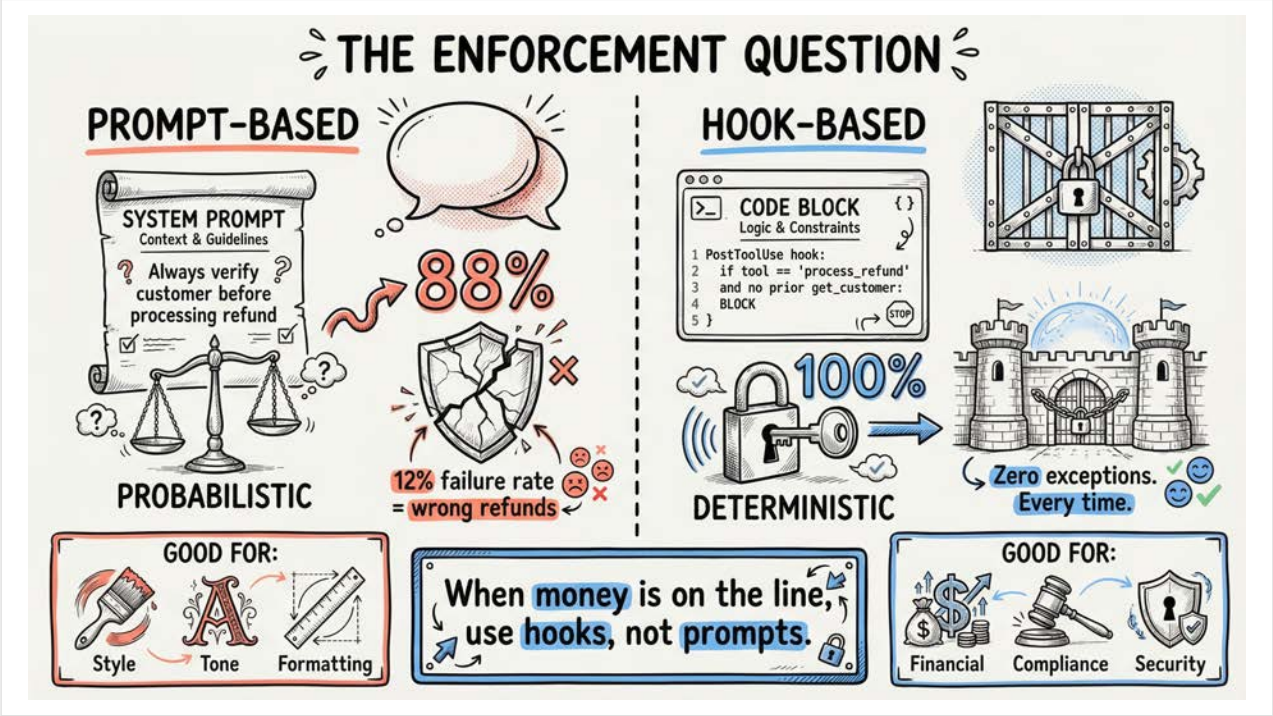
★ Subagents are isolated. If the coordinator doesn't pass it, the subagent doesn't know it.

Exam Tip: The exam has a question about narrow decomposition causing incomplete coverage.

DOMAIN 1: AGENTIC ARCHITECTURE (27%)

The Enforcement Question

When to use prompts vs hooks. The most important concept in the cert.



What You Need to Know

- Prompts are probabilistic -- they work most of the time, but not always. The exam gives a scenario with a 12% skip rate.
- Hooks are deterministic -- they physically block actions unless conditions are met. 100% enforcement.
- Prompts are best for style, tone, formatting -- things where occasional inconsistency is acceptable.
- Hooks are best for financial, compliance, security -- anything where one failure has real consequences.
- Think of prompts as suggestions and hooks as laws.

> **TL;DR**
>
> *At production scale, prompts break down. A well-crafted system prompt can still fail 1 in 1000 times -- and in financial or compliance contexts, that failure rate is unacceptable. Hooks exist precisely for those cases.*

★ When money is on the line, use hooks, not prompts.

Exam Tip: Q1 correct answer: A -- programmatic prerequisite that blocks downstream tools.

DOMAIN 2

TOOL DESIGN & MCP

How Claude connects to the outside world

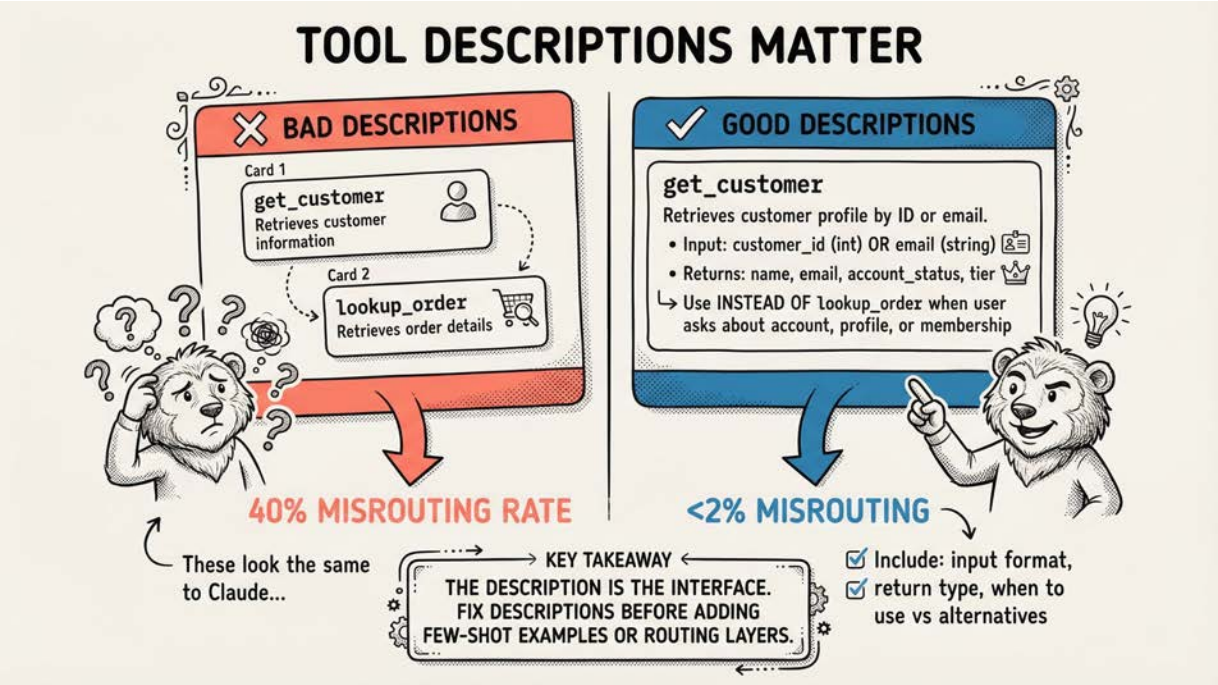
18%

of the exam

DOMAIN 2: TOOL DESIGN & MCP (18%)

Tool Descriptions Matter

The highest-leverage fix in the entire certification.



What You Need to Know

- Tool descriptions are how Claude decides which tool to call.
- Vague, overlapping descriptions cause frequent misrouting.
- Fix: include input format, return type, and state when to use this tool vs alternatives.
- Adding a 'do NOT use this for...' line dramatically reduces errors.
- Better descriptions beat few-shot examples, routing layers, or any other fix.

> **TL;DR**
>
> *Every wrong tool call costs tokens. Claude might reach the right answer eventually, but if it tried three wrong tools first, you paid for all of them. Better descriptions eliminate that waste at the source.*

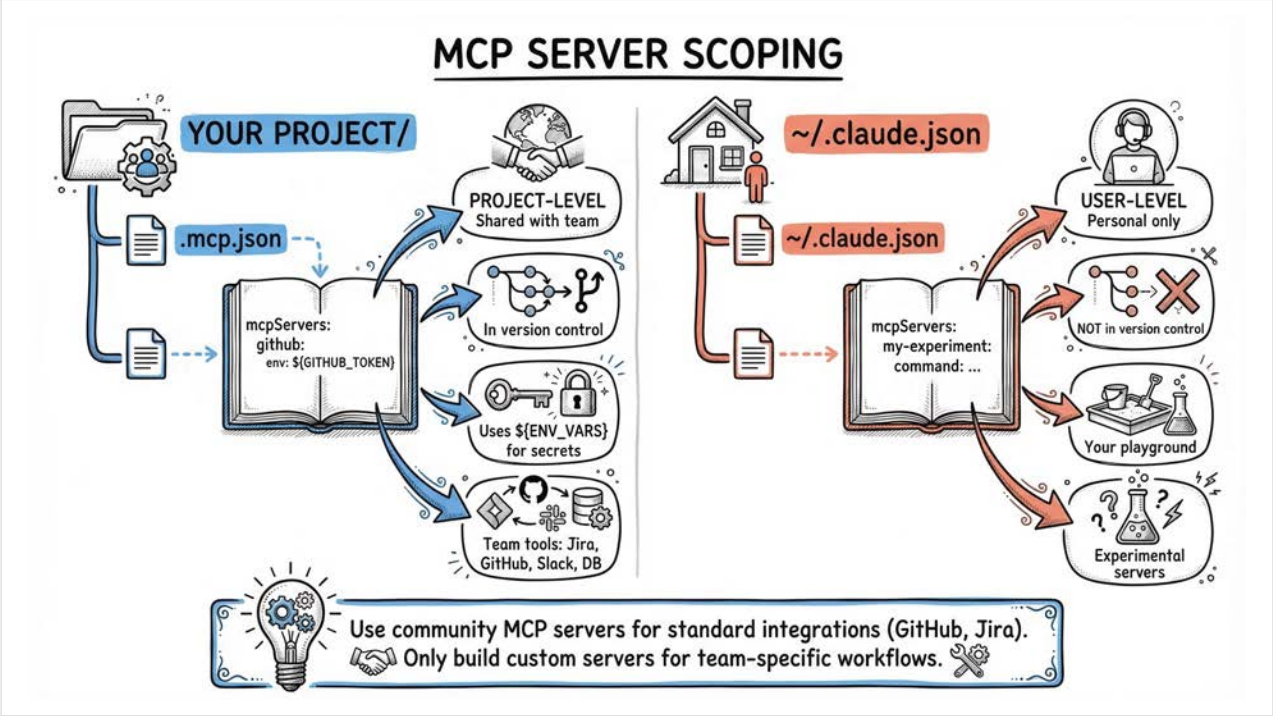
★ The description IS the interface. Fix descriptions before trying anything more complex.

Exam Tip: Q2 correct answer: B -- expand tool descriptions.

DOMAIN 2: TOOL DESIGN & MCP (18%)

MCP Server Scoping

Project-level vs user-level configuration.



What You Need to Know

- MCP is how Claude connects to external tools like GitHub, Slack, databases.
- Project-level (.mcp.json): shared with team via git. Uses environment variables for secrets.
- User-level (~/.claude.json): personal sandbox. Not shared. For experiments.
- Use community MCP servers for standard integrations. Only build custom for unique workflows.

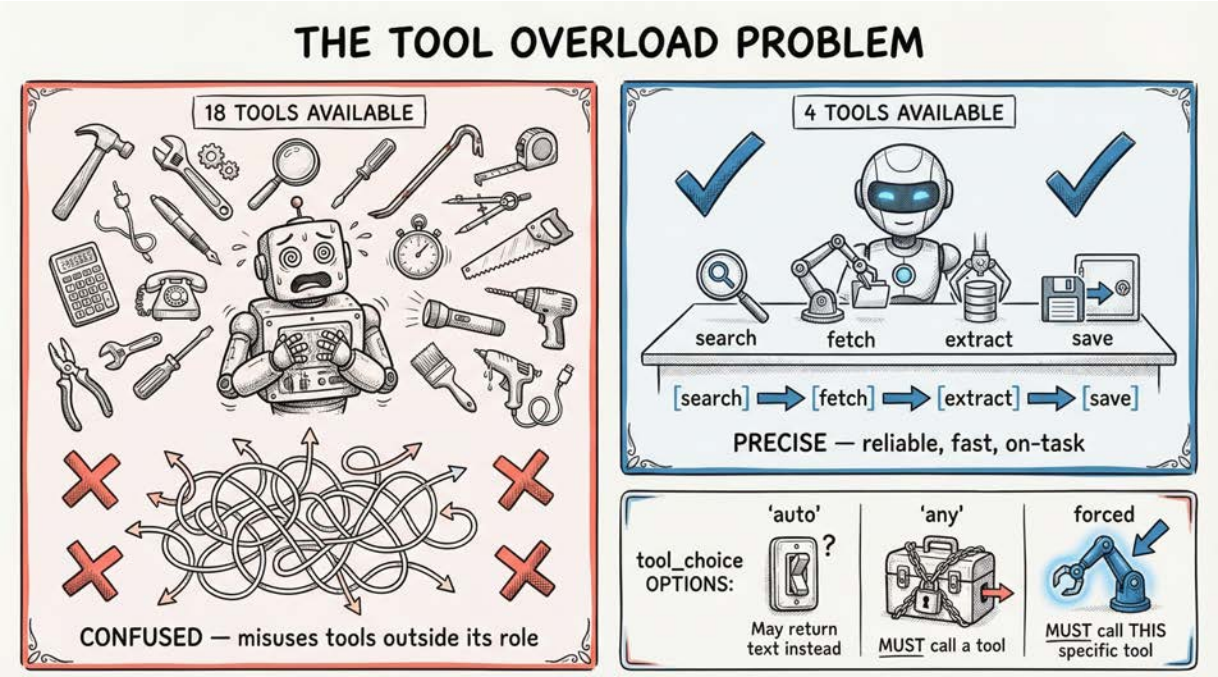
> **TL;DR**
>
> *MCP is the right tool for standardized integrations. For custom workflows, skills are more token-efficient. The scoping rules exist so team tools stay shared and personal experiments stay personal.*

★ Project MCP = team tools (shared). User MCP = your playground (personal).

DOMAIN 2: TOOL DESIGN & MCP (18%)

The Tool Overload Problem

Less choice means better decisions.



What You Need to Know

- 18 tools = confused agent. 4-5 focused tools = precise, reliable agent.
- tool_choice modes: 'auto' (Claude decides), 'any' (must use a tool), 'forced' (must use this specific tool).
- Pattern: force the right tool on step one, then switch to auto for the rest.
- Constraint makes agents precise.

> **TL;DR**
>
> *Fewer choices means better decisions. An agent with 4 focused tools just calls the right one. An agent with 18 spends cognitive budget figuring out which tool to use. Constraint is a feature, not a limitation.*

★ Keep agents to 4-5 tools max. An agent with 18 tools makes worse decisions than one with 5.

DOMAIN 3

CLAUDE CODE CONFIGURATION

CLAUDE.md, skills, commands, plan mode, CI/CD

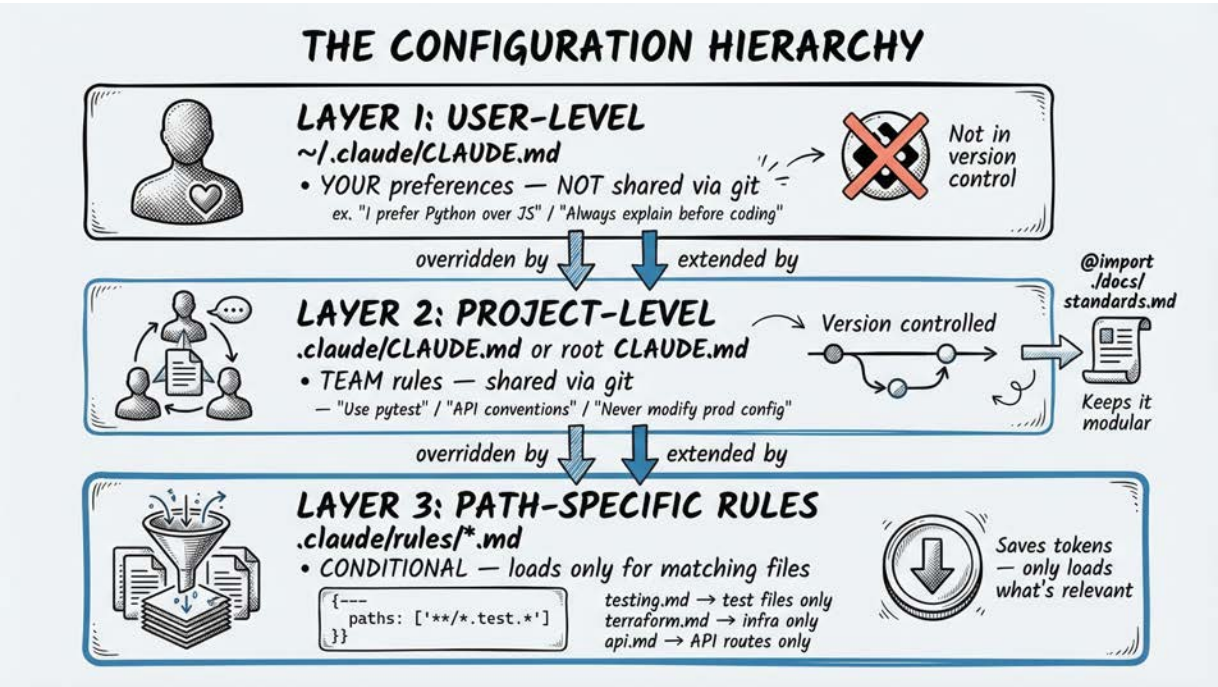
20%

of the exam

DOMAIN 3: CLAUDE CODE CONFIG (20%)

The Configuration Hierarchy

Three layers -- stop dumping everything in one file.



What You Need to Know

- User-level (~/.claude/CLAUDE.md): personal preferences. Not shared via git.
- Project-level (.claude/CLAUDE.md): team rules and conventions. Shared via version control.
- Path-specific (.claude/rules/*.md): conditional rules that only load for matching files.
- Example: testing rules only load when editing test files. API rules only load in the API folder.
- This saves tokens and keeps Claude focused on what's relevant.

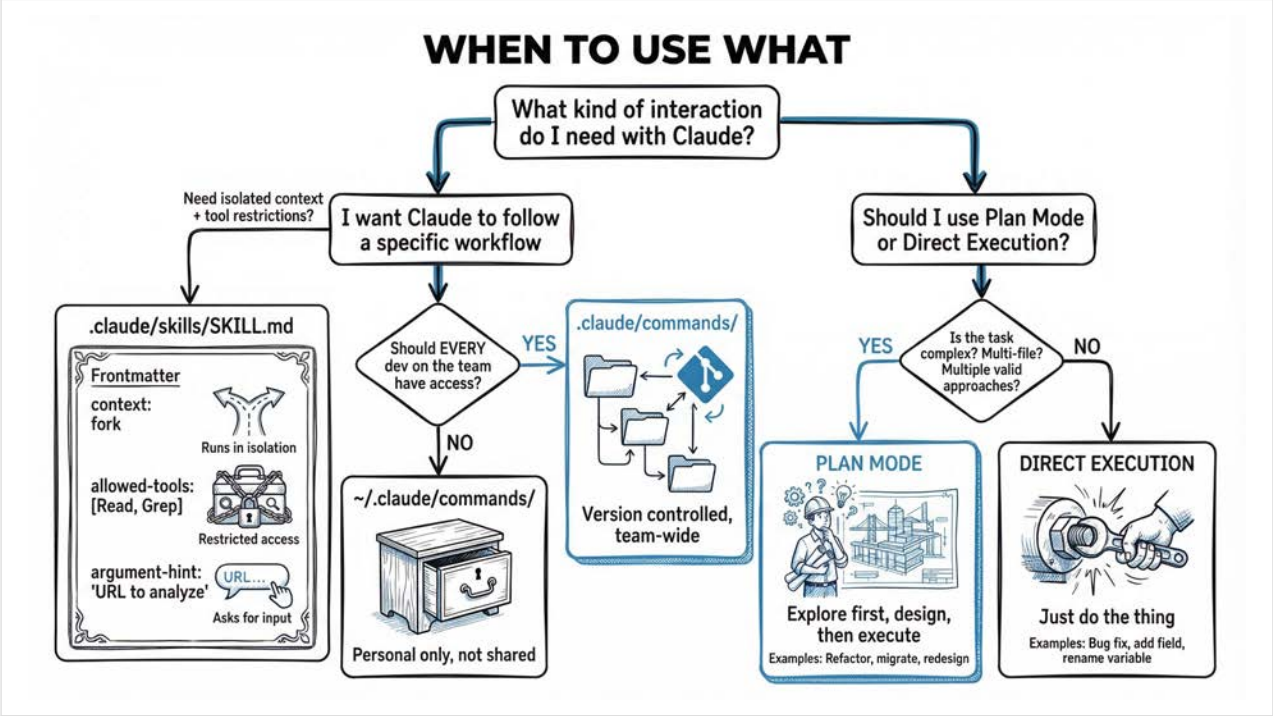
> **TL;DR**
>
> *CLAUDE.md is an instruction set, not a knowledge base. It gets injected into every session whether it's relevant or not. Path-specific rules fix this -- only loading what applies to the file you're currently editing.*

★ Split your monolith CLAUDE.md into three layers. Path-specific rules are the golden nugget.

DOMAIN 3: CLAUDE CODE CONFIG (20%)

When to Use What

Commands vs skills vs plan mode.



What You Need to Know

- Commands: reusable prompts triggered with /slash. Team-wide in .claude/commands/, personal in ~/.claude/commands/.
- Skills: runs in isolated context (context: fork). Defines allowed tools and can prompt for arguments.
- Plan mode: for complex, ambiguous, multi-file tasks. Claude explores without modifying anything.
- Direct execution: for simple, single-file fixes. Don't over-plan straightforward changes.

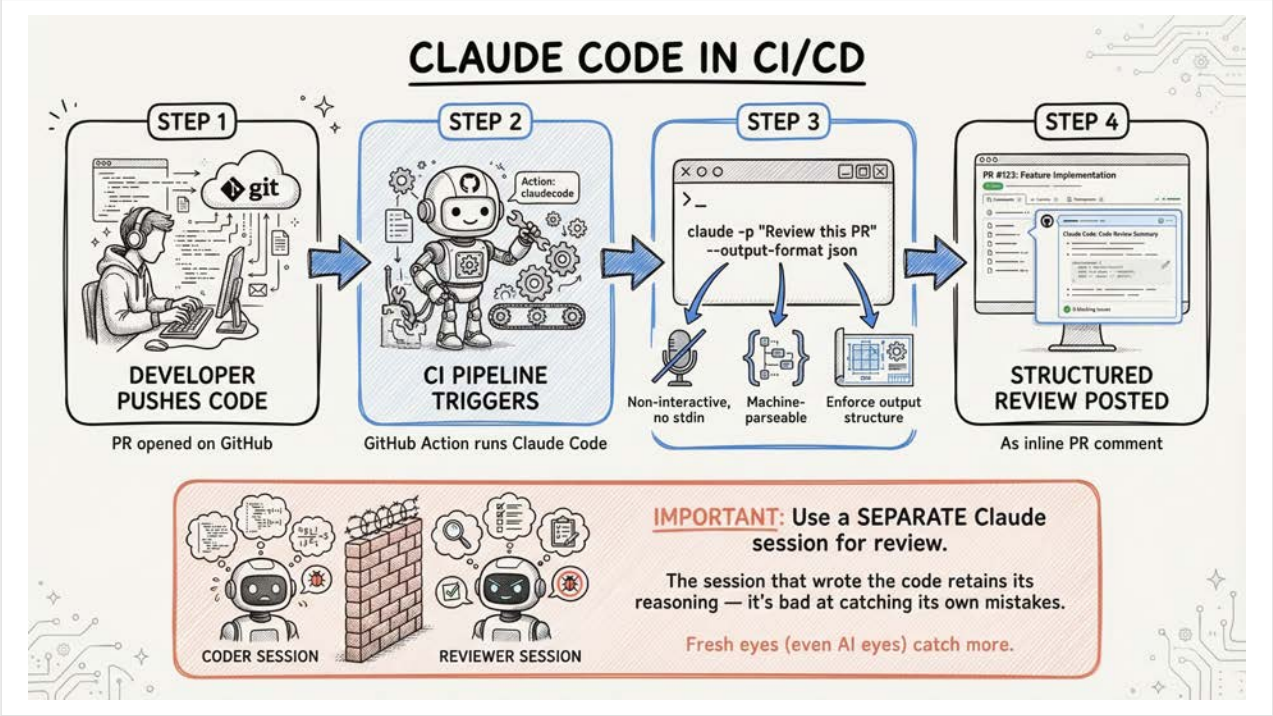
> **TL;DR**
>
> *All three tools -- commands, skills, plan mode -- share the same goal: keep the main conversation clean. Offload the messy work and bring back only the result.*

★ Match the tool to the task complexity. Skills isolate. Plan mode explores. Direct execution just does it.

DOMAIN 3: CLAUDE CODE CONFIG (20%)

Claude Code in CI/CD

Two flags turn Claude Code into an automation tool.



What You Need to Know

- -p flag: non-interactive. No prompts, no confirmations. Runs and returns the result.
- --output-format json: structured output that other tools can parse.
- Together, these let you trigger Claude Code from any CI/CD pipeline.
- Critical: use a SEPARATE session for review. Same session that wrote code is biased.
- Fresh eyes -- even AI eyes -- catch more.

> **TL;DR**
>
> *A model that generated code is biased toward thinking that code is correct. A fresh session with no prior reasoning about it is a genuinely independent reviewer. Use separate sessions for writing and reviewing.*

★ Write in one session, review in a different one. Self-review is inherently biased.

DOMAIN 4

PROMPT ENGINEERING

Making Claude's output reliable and consistent

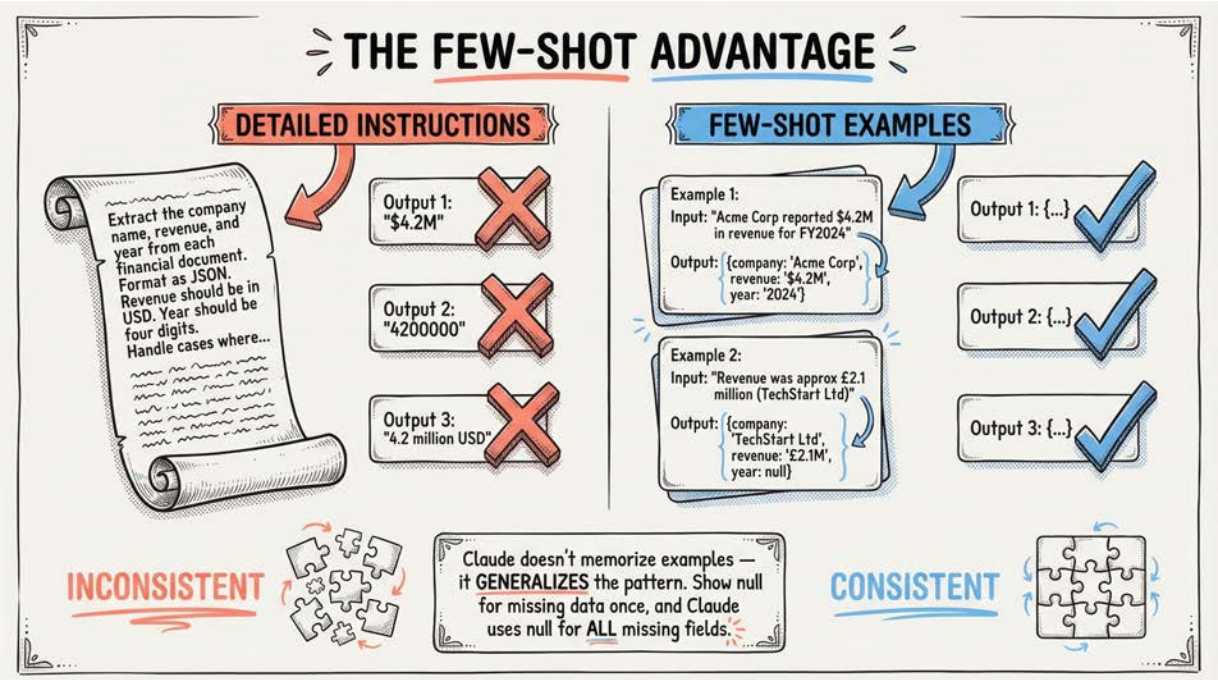
20%

of the exam

DOMAIN 4: PROMPT ENGINEERING (20%)

The Few-Shot Advantage

Why 2-3 examples beat a full page of instructions.



What You Need to Know

- When Claude gives inconsistent output, show it 2-3 examples instead of writing more instructions.
- Claude doesn't memorize -- it generalizes the pattern. Show null for missing data once, it uses null everywhere.
- Few-shot examples also reduce hallucination in extraction tasks.
- Two to three examples beat a paragraph of detailed instructions every single time.

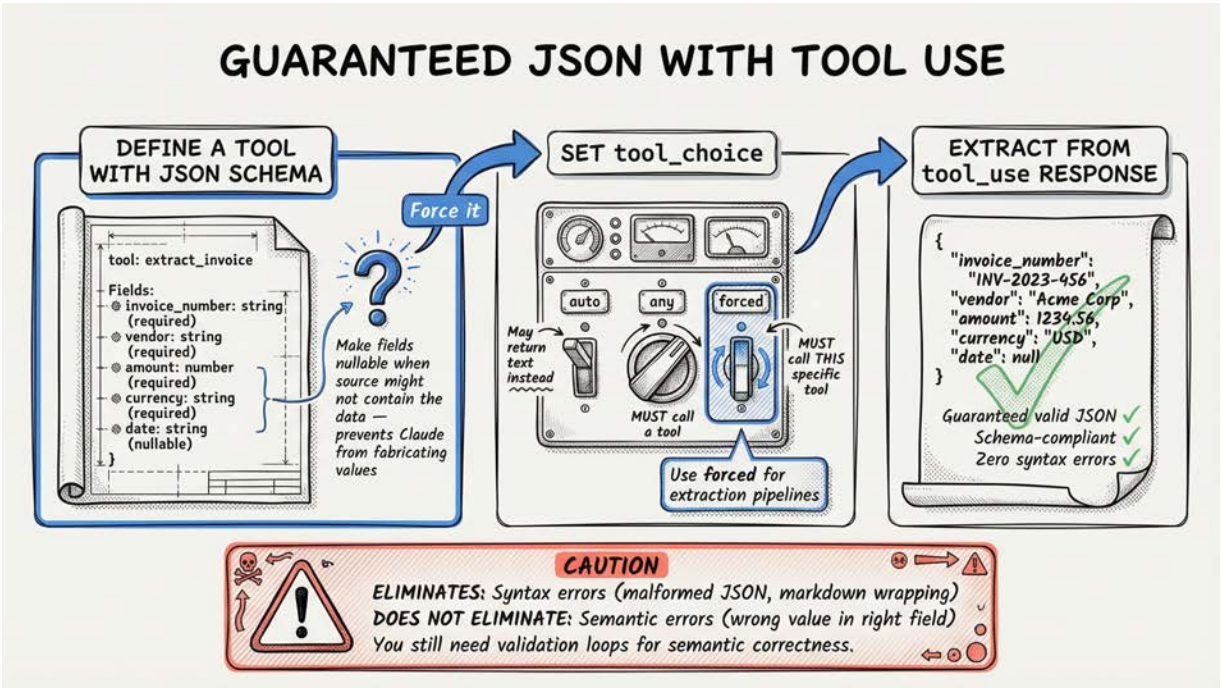
> **TL;DR**
>
> *Examples teach judgment, not just format. Claude generalizes from them -- handling edge cases you never explicitly covered. That's why two examples outperform a page of instructions.*

★ Show, don't tell. Examples teach Claude judgment, not just format.

DOMAIN 4: PROMPT ENGINEERING (20%)

Guaranteed JSON with tool_use

Structured output that never breaks.



What You Need to Know

- Define a tool with a JSON schema. Force Claude to call it with tool_choice.
- Make fields nullable when source data might be missing. Prevents Claude from fabricating values.
- This eliminates syntax errors (malformed JSON) but NOT semantic errors (wrong values).
- You still need validation loops for content correctness.

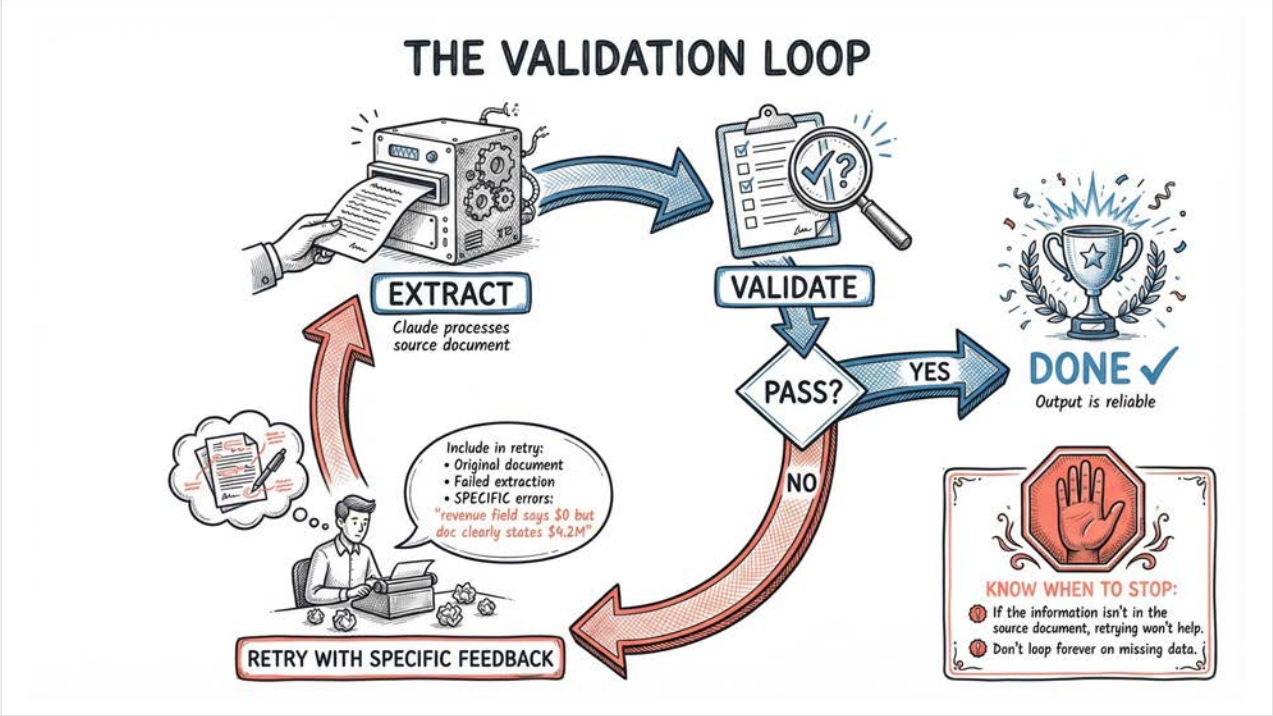
> **TL;DR**
>
> *Optional fields give Claude a safe way to express uncertainty. Required fields with missing data get fabricated. Nullable schema fields are your primary safeguard against hallucination in extraction tasks.*

★ tool_use + JSON schema = guaranteed valid JSON. But validate the values separately.

DOMAIN 4: PROMPT ENGINEERING (20%)

The Validation Loop

Extract, validate, retry with specific feedback.



What You Need to Know

- Pattern: extract, validate, retry with SPECIFIC error feedback. Not 'try again.'
- Good feedback: 'revenue says 0 but document clearly states 4.2 million on page 3.'
- Know when to stop: if the data isn't in the source, retrying won't produce it.
- Claude will start making things up if you push retries past the point of available information.

TL;DR

Validation is as much about knowing when to stop as it is about catching errors. Format errors are fixable with feedback. Missing information is not -- retrying won't produce data that isn't in the source.

★ Retry with specific errors. Stop when the data isn't in the source.

DOMAIN 5

CONTEXT MANAGEMENT

Keeping Claude sharp over long conversations

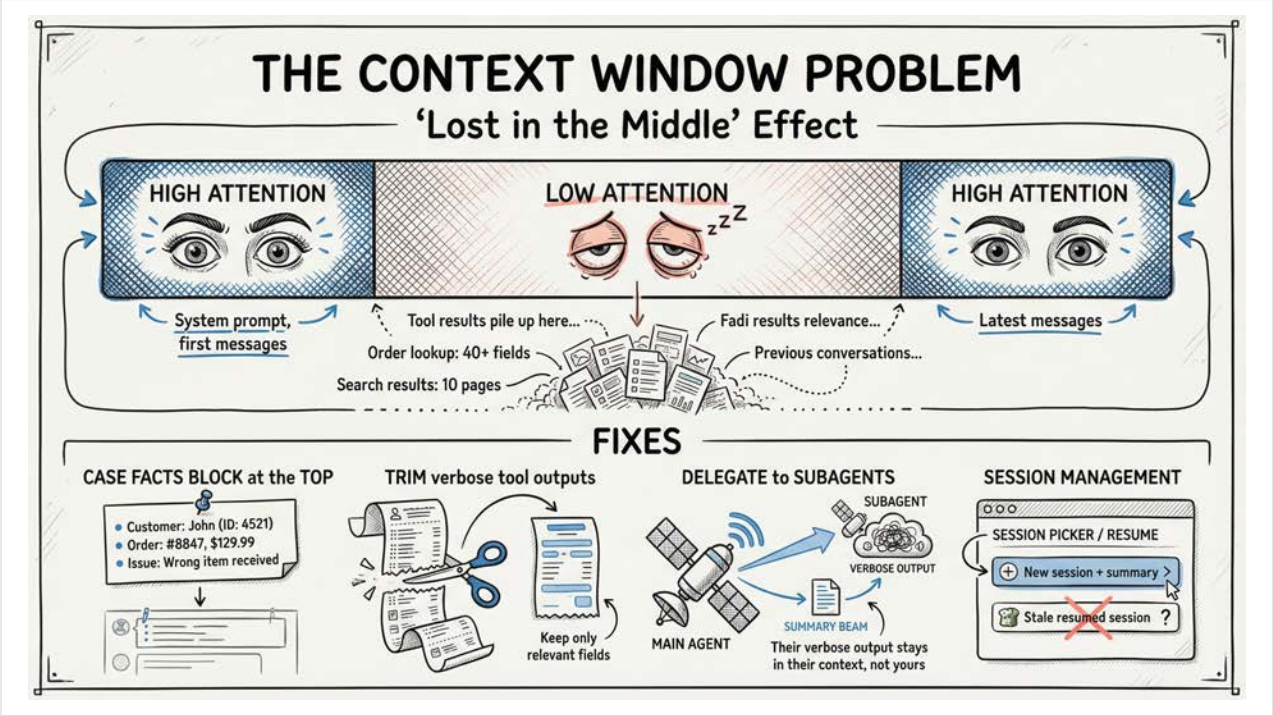
15%

of the exam

DOMAIN 5: CONTEXT MANAGEMENT (15%)

The Context Window Problem

Claude reads the start and end well. The middle gets fuzzy.



What You Need to Know

- 'Lost in the middle' effect: Claude processes beginning and end reliably, may miss middle.
- Every tool result adds data to the middle. 40 fields when you need 5.
- Fix 1: pin key facts at the TOP of context in a summary block.
- Fix 2: trim verbose tool outputs to only relevant fields.
- Fix 3: delegate verbose exploration to subagents. Their output stays in their context.
- A new session with a summary beats a stale, bloated conversation.

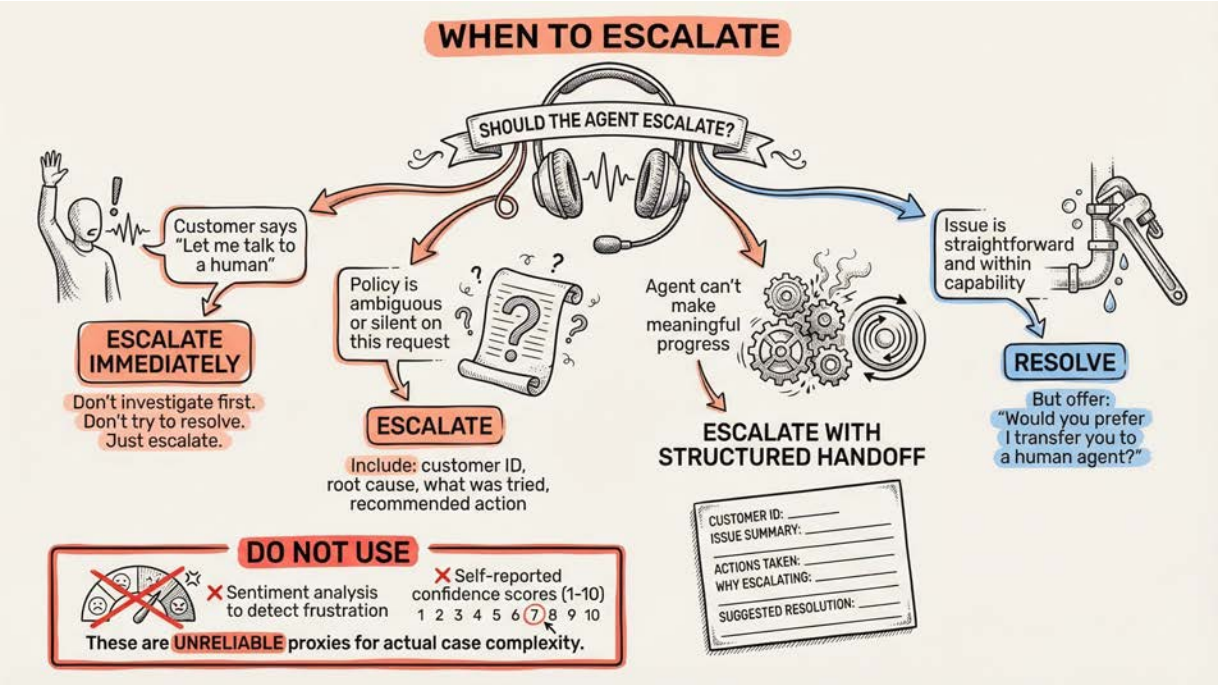
> **TL;DR**
>
> *Attention degrades over a long context window the same way it degrades on a long drive. The solution is not to push through -- it's to periodically reset. A new session with a summary is cleaner than a bloated conversation.*

★ Pin key facts at top. Trim tool outputs. Delegate heavy research to subagents.

DOMAIN 5: CONTEXT MANAGEMENT (15%)

When to Escalate

Agent-to-human handoff rules.



What You Need to Know

- Customer asks for a human: escalate immediately. No exceptions.
- Policy is ambiguous: escalate with full handoff package (customer ID, root cause, what was tried).
- Straightforward issue: resolve, but offer to transfer.
- Do NOT use sentiment analysis or confidence scores. Both are unreliable proxies.

TL;DR

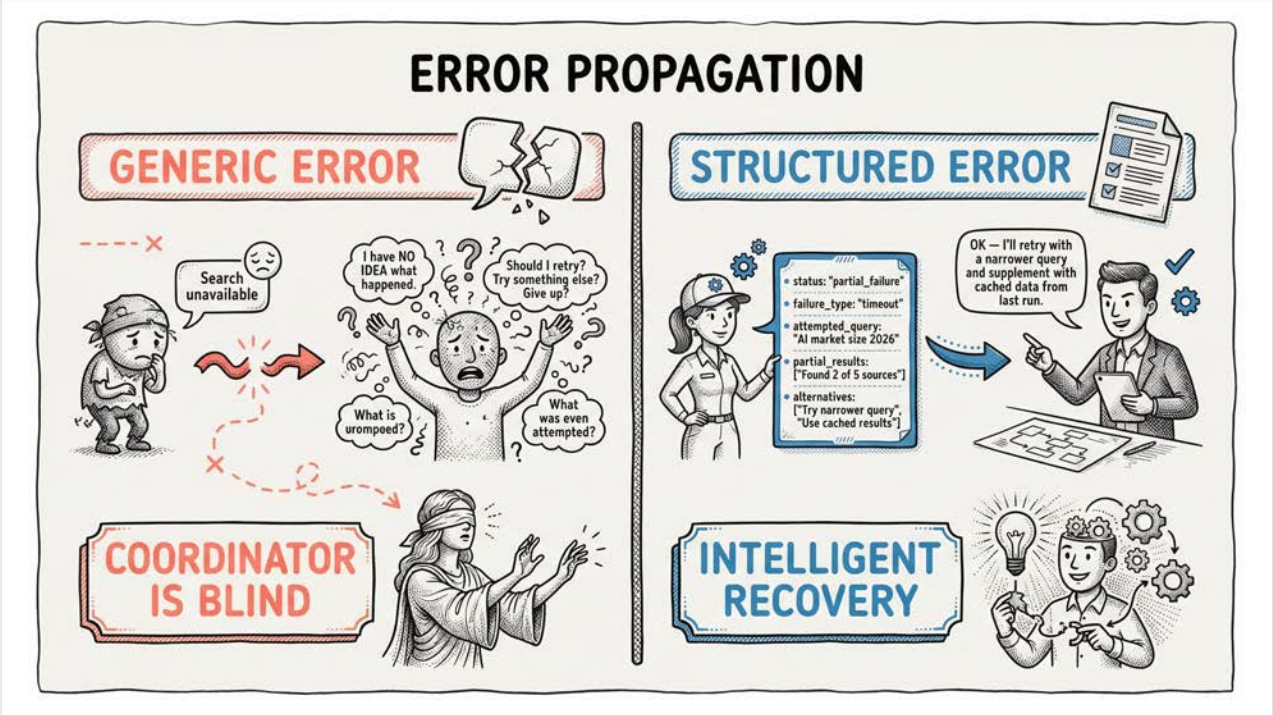
Respecting an explicit request for a human is not a failure -- it is the correct outcome. Sentiment and confidence scores misfire on sarcasm, cultural differences, and edge cases. Only explicit signals are reliable escalation triggers.

★ If they ask for a human, give them a human. No exceptions.

DOMAIN 5: CONTEXT MANAGEMENT (15%)

Error Propagation

Generic errors blind the coordinator. Structured errors enable recovery.



What You Need to Know

- 'Search unavailable' tells the coordinator nothing. It can't make intelligent recovery decisions.
- Structured errors include: what failed, what was attempted, partial results, alternatives.
- Now the coordinator can retry, use cached data, switch sources, or note the gap.
- Silent suppression and total termination are both anti-patterns.

> **TL;DR**
>
> *Graceful failure means the system can recover from what went wrong. A generic error is a dead end. A structured error with context, partial results, and alternatives gives the coordinator enough to make a smart next move.*

★ Never return a generic error. Include what broke, what was tried, and what could be done instead.

THE 5 RULES

Print this page. Pin it up. These patterns appear across all 5 domains.

1 HOOKS > PROMPTS

When compliance must be 100%, use programmatic enforcement. Prompts are suggestions. Hooks are laws.

2 STRUCTURED ERRORS ALWAYS

Never return generic 'failed.' Include what broke, what was tried, what partially worked, what alternatives exist.

3 SCOPE YOUR TOOLS

4-5 tools per agent, not 18. An agent with fewer, focused tools makes dramatically better decisions.

4 INDEPENDENT REVIEW

A second Claude instance catches what the first won't. Self-review is biased by prior reasoning.

5 EXAMPLES > INSTRUCTIONS

2-3 concrete few-shot examples beat a paragraph of detailed prose instructions every single time.

These aren't five separate ideas. They're the same principle: be explicit, be structured, and don't trust 'probably works' for things that must work every time.

RESOURCES

Keep learning beyond this guide.

Official Certification Exam Guide

The original 40-page Anthropic document this study guide is based on. Read it alongside this guide for full depth.

`Download the official exam guide ■`

Interactive Study Prompts

Community-created prompts by @hooeem on X. Paste them into Claude Code and it interviews you domain by domain, adapting to your level.

`View the X post with all prompts ■`

Claude Code Documentation

Official Anthropic docs for CLAUDE.md, skills, hooks, MCP, plan mode, and all CLI flags.

`docs.anthropic.com/en/docs/claude-code ■`

Mark's Community

Join 800+ builders inside Early AI Dopters for coaching, courses, and every resource from this channel.

`skool.com/earlyaidopters ■`